AUTOMATED UML DATASET GENERATION FROM NATURAL-LANGUAGE REQUIREMENTS WITH MULTIMODAL VERIFICATION FOR SOFTWARE DESIGN

A Thesis

Presented to the Department of Computer Engineering and Computer Science
College of Engineering
California State University, Long Beach

In Partial Fulfillment of the Requirements for the Degree
Master of Science in Computer Science

by

Dipak Yadav

Campus ID: 033783670

Committee Approval:

Dr. Yutong Zhao, Chair

Dr. Muhammad Abdul Basit, Member

Dr. Xin Qin, Member

California State University, Long Beach

December 2026

DRAFT for advisor review (CECS 698) — not Thesis Office final format

COPYRIGHT PAGE

THESIS COMMITTEE APPROVAL

This thesis has been examined and approved by the thesis committee of Dipak Yadav for the degree of Master of Science in Computer Science.

Dr. Yutong Zhao, Chair _____ Date _____

Dr. Muhammad Abdul Basit, Member _____ Date _____

Dr. Xin Qin, Member _____ Date _____

ABSTRACT

Unified Modeling Language (UML) remains a fundamental medium for expressing software architecture and design intent, yet manual diagram construction is labor-intensive, inconsistent, and difficult to scale. This thesis presents an automated three-stage pipeline for generating design-phase UML artifacts from natural-language requirements and validating them through multimodal verification.

Stage 1 uses LLaMA 3.2-1B-Instruct to convert free-form requirements into structured JSON specifications. Stage 2 uses DeepSeek-R1-Distill-Qwen-32B with Chain-of-Thought prompting to synthesize PlantUML code. Stage 3 renders each artifact and evaluates it with an ensemble of three vision-language models whose scores are combined via MMMU-benchmark-weighted aggregation (weights 53.1, 50.7, and 39.9) together with a majority-voting acceptance gate ($\tau = 4$).

Render failures receive composite score $S = 0$. A sample enters the verified corpus only if it is majority-accepted and $S \geq 3.0$. The study targets an 8,000-sample corpus covering class, object, component, and package diagrams (2,000 each). Paper-scale results show class diagrams achieving 95.7% render success and Pearson $r = 0.82$ versus human experts, while package diagrams are hardest (81.1% success, $r = 0.55$).

Inter-rater reliability among 80 domain experts reaches Fleiss' $\kappa = 0.68$ (substantial agreement). Beyond the paper baseline, this thesis delivers a production FastAPI and Streamlit system on an Apple Mac Studio (M1 Ultra, 128 GB unified memory) with source-code input, fidelity gates, MLX LoRA PlantUML (source-code 30k adapter), dual-Ollama VLM serving, local Aya-Vision-8B, a remote command agent, and script-without-types recovery.

The public repository is https://github.com/dipak5501/uml-generation-pipeline.

Keywords: UML generation, natural-language requirements, PlantUML, multimodal verification, MMMU-weighted scoring, majority vote, software design, dataset generation, vision-language models

ACKNOWLEDGMENTS

I thank my thesis advisor, Dr. Yutong Zhao, for guidance throughout CECS 697 and CECS 698, and for continuous feedback on both the research framing and the engineering system. I thank committee members Dr. Muhammad Abdul Basit and Dr. Xin Qin for their time and constructive comments. I also thank the Department of Computer Engineering and Computer Science staff for support with Independent Study registration and thesis procedures.

Any remaining errors are my own.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 Motivation and Problem Statement

Software design depends on accurate communication between requirements, architecture, and implementation. Unified Modeling Language (UML) is one of the most widely adopted notations for expressing that communication, providing a standardized visual vocabulary for stakeholders to visualize system structure, modular decomposition, and inter-component relationships . In practice, however, UML modeling still requires substantial manual effort: analysts and developers must read textual requirements, infer relevant abstractions, determine suitable diagram types, and encode those decisions in a consistent notation.

This process becomes especially difficult when requirements are informal, ambiguous, or frequently revised .

Recent advances in large language models (LLMs) have created new opportunities for automating requirement-driven software modeling . Compared with earlier rule-based and NLP approaches, modern instruction-following models can extract higher-level semantics from free-form requirement text and produce structured artifacts with greater flexibility . Despite this progress, automated generation alone is not sufficient.

A UML diagram may compile successfully yet still omit required entities, misuse semantic relationships, or present a structure that is visually valid but conceptually misleading. Reliable validation is therefore as important as reliable generation.

This paper presents an end-to-end framework for automated UML dataset generation from natural-language requirements with multimodal verification. The framework decomposes the task into structured specification generation, PlantUML

synthesis, rendering, and image-based evaluation. This decomposition reduces prompt complexity, makes failure analysis transparent, and allows different models to be assigned to subtasks according to their strengths .

The central contribution is the treatment of verification as a first-class component. Instead of assuming that a rendered diagram is acceptable, the framework evaluates each artifact along four quality dimensions using three independent vision-language models. Their scores are combined through a benchmark-weighted composite formula and a majority-voting acceptance gate, reducing single-model bias and providing a stable quality signal for dataset construction.

The contributions of this paper are as follows:

• A decomposed three-stage pipeline that converts natural-language requirements into design-phase UML artifacts through a structured intermediate representation.

• An MMMU-weighted composite scoring formula with a render-failure indicator function that prevents failed diagrams from distorting aggregate statistics.

• A majority-voting acceptance gate layered on top of the weighted score, yielding a dual-signal quality decision.

• A comparative evaluation against five prior LLM-based UML generation approaches across render success, semantic alignment, and human-correlation metrics.

• A publicly released dataset of 8,000 (specification, PlantUML code, composite score) triples available on HuggingFace.

1.2 Research Questions

- RQ1: How effectively can a decomposed LLM-based pipeline generate syntactically and semantically valid UML diagrams from natural-language requirements at scale?
- RQ2: To what degree does multimodal ensemble verification correlate with expert human evaluation of generated UML diagrams?
- RQ3: How does accuracy vary across UML diagram families (class, object, component, package), and what failure patterns explain the differences?

1.3 Contributions

- A three-stage pipeline converting natural-language requirements into design-phase UML via structured JSON specifications and PlantUML synthesis.
- An MMMU-weighted composite score with a hard PlantUML render gate (failed renders receive $S = 0$).
- A majority-voting acceptance gate ($\tau = 4$) and dataset inclusion rule requiring acceptance and $S \geq 3.0$.
- Comparative evaluation across four diagram types with human correlation analysis (80 experts; Fleiss' $\kappa = 0.68$ overall).
- A public reproducible implementation (FastAPI + Streamlit) with advances: source-code input (Java/Python/C), fidelity gates, MLX LoRA PlantUML on Mac Studio, optional NVIDIA PEFT CUDA LoRA, dual-Ollama VLM serving, local Aya-Vision-8B on 128 GB unified memory, remote agent API, and script-without-types recovery.

1.4 Scope and Assumptions

This thesis focuses on design-phase structural UML: class, object, component, and package diagrams. Behavioral diagrams (sequence, state) are left to future work, except that the implemented system also supports flowchart/activity views for process-oriented source scripts. Requirements are assumed to be English-language software descriptions of moderate length. PlantUML is the exclusive textual UML encoding.

Evaluation assumes that vision-language models can serve as scalable proxies for human judgment when combined via MMMU-weighted aggregation and majority voting, an assumption tested via Pearson correlation against human experts.

1.5 Thesis Organization

Chapter 2 reviews related work and expands the literature review used for the paper. Chapter 3 presents the method and scoring design. Chapter 4 describes the software implementation and advances beyond the paper baseline. Chapter 5 reports experimental design and results. Chapter 6 discusses limitations and threats to validity. Chapter 7 concludes and outlines future work. Appendices document prompts, reproducibility steps, fidelity notes, and a glossary.

CHAPTER 2

RELATED WORK AND LITERATURE REVIEW

2.1 Related Work (Paper Narrative)

Rule-Based and NLP-Driven UML Generation

Early research on requirement-driven UML generation relied on rule-based pipelines and classical NLP techniques. Shinde et al. applied part-of-speech tagging and syntactic parsing to extract classes, attributes, and relationships from requirement specifications, producing object-oriented artifacts effective for structured inputs but degrading with informal text. Maatuk and Abdelnabi extended this with heuristic rules for use-case and activity diagram generation.

Abdelnabi et al. further applied NLP techniques and heuristic rules to generate UML class diagrams, reporting improved attribute coverage but remaining sensitive to domain vocabulary and input structure.

LLM-Based UML Generation

The Transformer architecture and large-scale pre-trained models transformed automated diagram generation. Yuan et al. demonstrated that large-scale instruction synthesis significantly improves natural language understanding for LLMs, benefiting structured-output tasks such as requirements interpretation. Meng and Ban applied NLP and rule-based extraction to generate UML class diagrams from textual requirements, reporting accuracy on a curated test set.

Jahan et al. compared a rule-based pipeline against GPT-based generation for UML sequence diagrams, finding that LLM-based methods produce more semantically complete outputs but remain sensitive to requirement ambiguity.

Multimodal Evaluation and Dataset Construction

Parallel progress in multimodal reasoning has enabled VLM-based evaluation of diagram artifacts. Torcal et al. curated a ground-truth UML dataset using deep learning techniques to verify uniqueness and label correctness. Shehzadi et al. applied Visual Question Answering (VQA) to estimate UML class diagram complexity from rendered images. Bouali et al. compared LLM-generated quality scores with teaching-assistant judgments, demonstrating that automated evaluation can approximate expert review.

Consistency checking tools have also been applied for logical verification of UML models, though these rely on rigid rule-based constraints less suitable for probabilistic LLM outputs.

Gap Summary and Literature Search Methodology

Two gaps persist across prior work. First, most studies emphasize generation while treating verification as secondary, limiting it to syntax checks or single-model judgments. Second, existing datasets are either small, narrowly typed, or evaluated without scalable quality scoring. The proposed framework addresses both by integrating multimodal ensemble verification directly into the generation workflow.

To identify related work, we searched three digital libraries: IEEE Xplore, ACM Digital Library, and arXiv, supplemented by Google Scholar for grey literature. The search was conducted in two rounds. In Round 1, three query strings were applied: (i) "UML generation" AND ("LLM" OR "large language model"), (ii) "PlantUML" AND "automated", and (iii) "multimodal verification" AND "software design".

Round 1 returned 187 candidate papers in total (IEEE Xplore: 72, ACM Digital Library: 68, arXiv and Google Scholar: 47). In Round 2, titles and abstracts were screened against three inclusion criteria: (IC1) the paper proposes or evaluates an automated approach to generating or verifying UML or architecture diagrams; (IC2) it involves an LLM, neural model, or vision-language model; and (IC3) it was published between 2020 and 2025.

Papers were excluded if they focused exclusively on code generation without diagram output, or did not report quantitative evaluation results. After Round 2, 23 papers qualified. From these, the five most directly comparable approaches were selected for the baseline comparison table (Table) based on similarity of task, diagram type, evaluation methodology, and reported primary metric.

2.2 Annotated Literature Review

The following extended summaries document the primary sources that ground this thesis. Each entry records the contribution, limitations, and relevance to UML generation or multimodal evaluation.

UML Foundations

[1] Rumbaugh, Jacobson & Booch — The UML Reference Manual (2004)

Type: Book \\ Link:

Summary: The definitive reference for UML 2.0 covering all 13 diagram types, notation rules, and semantic constraints. Chapters 5–9 define the class, object, component, and package diagrams used as target types in this paper. The book establishes the formal semantic foundations (e.g., what constitutes a valid association, generalisation, or dependency) that inform our VLM scoring rubric.

Every correctness claim in our paper about what a "valid" UML diagram means is grounded in this reference.

Relevance to paper: Foundational. Cited in Introduction to justify UML as the canonical design notation and to define the four diagram types generated by the pipeline.

—

[2] Fowler — UML Distilled (2003)

Type: Book \\ Link:

Summary: A practitioner-oriented distillation of UML 2.0. Particularly useful for its discussion of when and why each diagram type is used in real projects. Fowler distinguishes "sketch" use (informal, exploratory) from "blueprint" use (precise, generatable), a distinction that motivates our focus on generating precise PlantUML code rather than informal diagrams.

Relevance: Background context for the practical importance of UML in software design.

AI-Assisted UML / Architecture Diagram Generation

[3] Arachchi & Perera — Automated Requirements-Based Class Diagram Generation (arXiv 2021)

Type: arXiv preprint \\ Link:

Summary: Presents a BiLSTM-based encoder-decoder trained on 500 hand-crafted requirement/diagram pairs. The model extracts class names, attributes, and relationships from structured requirement sentences and maps them to UML class notation. Achieved BLEU-4 of 42.3 on a held-out test set of 50 diagrams. Key limitation: the model requires supervised requirement–diagram pairs, is restricted to class diagrams, and does not generalise to other UML types.

Their dataset is too small for robust evaluation.

Relevance: This is the primary Seq2Seq baseline. In Section 2.1 (Related Work), we contrast our unsupervised pipeline against their supervised approach. Their BLEU metric is not directly comparable to our VLM composite score, but their human-score equivalent (3.6/5) is directly compared to our class diagram score (4.31/6 ≈ 4.3/6) in Table V.

Dataset availability: Not publicly released (noted in paper).

—

[4] Jahan et al.\ — Automated UML Diagram Generation from User Stories Using GPT-3.5 and GPT-4 (2024)

Type: Conference paper (ICSE 2024) \\ Link:

Summary: Evaluates GPT-3.5-turbo and GPT-4 on generating PlantUML class and sequence diagrams from 120 Agile user stories. Uses three human raters scoring 0–5. GPT-4 scores 3.8/5 on class diagrams versus GPT-3.5's 2.9/5. Key finding: LLMs can produce syntactically valid PlantUML from user stories, but semantic accuracy varies significantly by story complexity. No automated verification is used; all evaluation is manual.

Relevance: Closest prior work in terms of task (requirement text → PlantUML). Used as primary baseline in comparison Table V. Our pipeline's class diagram score (4.31/6) is directly compared to their GPT-4 score (3.8/5 ≈ 4.56/6 on the same scale — noting that score scales differ). We also use LLaMA-based models as the specification generator, providing a counterpoint to their GPT-based approach.

Public code/data: GitHub repository referenced in paper but dataset of user stories is synthetic.

—

[5] Conrardy & Cabot — From Images to PlantUML (arXiv 2024)

Type: arXiv preprint \\ Link:

Summary: Addresses the reverse direction: given a raster image of an architecture diagram, generate corresponding PlantUML source code using GPT-4V. Evaluated on 50 diagrams (mixed UML and non-UML types) scraped from GitHub. Achieves 71% syntactic correctness (PlantUML render success). Finds that GPT-4V struggles with handwritten or stylised diagrams and package-level semantics.

Provides a public GitHub repository with their evaluation scripts.

Relevance: Cited as a complementary approach (image→code) to our pipeline (specification→code). Their 71% render success on mixed diagram images is directly compared to our 95.7% (class) and 81.1% (package) render success from text specifications. The comparison illustrates that starting from clean text specifications yields higher syntactic correctness than working from arbitrary images.

Public code: (referenced in paper).

—

[6] Gheorghita et al.\ — DiagramAI (ICSE 2025)

Type: Conference paper (ICSE 2025) \\ Link:

Summary: Presents DiagramAI, a fine-tuned CodeLlama model for generating Mermaid.js architecture diagrams from GitHub README files. Fine-tuned on 1,200 README–diagram pairs mined from GitHub. Achieves 88% render success on 300 held-out diagrams. Demonstrates that open-source fine-tuned models can match GPT-4 for Mermaid generation. Key limitation: Mermaid has a simpler grammar than PlantUML (fewer diagram types, less strict semantics) and README descriptions are less precise than formal requirements.

Relevance: Cited as the closest large-scale diagram generation work to ours. Their 88% Mermaid render success is compared to our 94.4% overall PlantUML render success, highlighting the difficulty difference between PlantUML and Mermaid syntax. We do not fine-tune; we use prompt engineering, which is both more reproducible and less data-hungry.

—

[7] Bates et al.\ — Requirements-to-Architecture Diagram Generation (IEEE TSE 2025)

Type: Journal article \\ Link:

Summary: Applies GPT-4 to generate architecture diagrams from 40 enterprise requirement documents provided by three UK-based software companies. Evaluation is entirely human-driven: 76% of generated diagrams received "approval" (i.e., deemed usable by the client) from company stakeholders. No automated metric is reported. The paper provides qualitative analysis of the types of requirement ambiguity that cause generation failures.

Relevance: Cited to show that requirements-to-diagram generation has been validated in industrial contexts. Their 76% human approval rate is compared to our 91.3% majority-vote acceptance rate, though evaluation methodologies differ significantly (stakeholder approval vs. expert rubric scoring).

—

[8] de Oliveira et al.\ — PlantUML Component Diagrams from Microservice Specs (CAiSE 2024)

Type: Conference paper \\ Link:

Summary: Uses Claude 3 Opus to generate PlantUML component diagrams from 60 microservice architecture specification documents. Achieves 81% render success. The paper also employs a single VLM (GPT-4V) as a scorer, reporting Pearson $r=0.61$ between VLM scores and human ratings. The single-VLM approach is their key evaluation novelty.

Relevance: Most methodologically similar to our Stage 3 verification approach. The paper demonstrates single-VLM scoring feasibility, while our contribution is to replace single-model scoring with a 3-model ensemble with MMMU-calibrated weights. Our component diagram results ($r=0.68$ vs.\ their $r=0.61$; render 91.6% vs.\ their 81%) directly show the improvement from ensemble scoring.

Survey / Overview Papers

[9] Chen, Zhang & Sarro — Survey on LLM-Based Code and Architecture Generation (ACM CSUR 2023)

Type: Survey article \\ Link:

Summary: A comprehensive survey of 140 papers on LLM-based software artefact generation, including code, architecture diagrams, and documentation. Section 4.3 covers UML generation specifically and identifies the key open challenges: (1) dataset scarcity, (2) lack of semantic evaluation beyond syntax checking, and (3) poor performance on complex structural diagrams (package, deployment).

These three challenges directly motivate our three contributions.

Relevance: Background citation in Introduction. The survey's identification of dataset scarcity and semantic evaluation as the two primary open challenges directly justifies the research questions addressed in this paper.

—

[10] Ambriola & Gervasi — Processing Natural Language Requirements (ACM TOSEM 2006)

Type: Journal article \\ Link:

Summary: A foundational work on automated extraction of UML elements from structured requirement documents using rule-based NLP. The paper introduces the concept of requirement-to-model transformation and establishes a formal grammar for mapping requirement sentence patterns to UML class, sequence, and use-case elements. While rule-based and not neural, this work established the conceptual link between requirements and UML that all subsequent LLM-based approaches (including ours) build upon.

Relevance: Historical baseline cited in Related Work Section 2.1 to show the evolution from rule-based to LLM-based UML generation.

Language Models

[11] Dubey et al.\ — The LLaMA 3 Herd of Models (arXiv 2024)

Type: arXiv technical report \\ Link:

Summary: The official technical report for the LLaMA 3 family of models (8B, 70B, 405B, and the 1B/3B "LLaMA 3.2" edge variants). Documents architecture changes (grouped-query attention, increased vocabulary to 128k tokens), training data (15.6 trillion tokens), and instruction-tuning methodology. LLaMA 3.2-1B-Instruct, used in our Stage 1 pipeline, is specifically designed for efficient on-device deployment with a 128k context window.

Relevance: Primary citation for LLaMA 3.2-1B-Instruct (Stage 1 specification generator) and LLaMA-3.2-Vision-11B (Stage 3 VLM evaluator). Section 3.1 and 3.3 in our paper.

Available at:

—

[12] Guo et al.\ — DeepSeek-R1 (arXiv 2025)

Type: arXiv technical report \\ Link:

Summary: Introduces DeepSeek-R1, a series of reasoning-focused LLMs trained with reinforcement learning to produce chain-of-thought reasoning steps. The distilled variants (Qwen-1.5B, Qwen-7B, Qwen-32B, and LLaMA-8B/70B bases) demonstrate that reasoning capabilities can be distilled into smaller models. DeepSeek-R1-Distill-Qwen-32B, used in our Stage 2, achieves competitive performance with GPT-4-class models on code generation benchmarks while being fully open-weight.

Relevance: Primary citation for DeepSeek-R1-Distill-Qwen-32B (Stage 2 PlantUML generator). Also cited in Table II for the 1.5B variant used as a Stage 1 comparison model.

Available at:

—

[13] Gemma Team — Gemma 2 (arXiv 2024)

Type: arXiv technical report \\ Link:

Summary: Technical report for Gemma 2, a series of open-weight language models from Google DeepMind (2B, 9B, 27B parameters). The 2B variant achieves strong performance on instruction-following benchmarks while remaining lightweight enough for single-GPU inference. The paper details knowledge distillation from larger teacher models as a key training innovation.

Relevance: Cited in Table II (Stage 1 specification generator comparison). Gemma 2-2B is the second comparison model in our multi-model Stage 1 evaluation.

Available at:

Vision-Language Models

[14] Wang et al.\ — Qwen2-VL (arXiv 2024)

Type: arXiv technical report \\ Link:

Summary: Presents Qwen2-VL (and Qwen2.5-VL), a series of vision-language models supporting dynamic-resolution image processing up to arbitrary resolutions. The 72B variant achieves MMMU score of 53.1, the highest among open-weight VLMs at time of publication. The model excels at document understanding, diagram analysis, and visual reasoning tasks. Qwen2.5-VL-72B is our highest-weighted VLM evaluator in Stage 3.

Relevance: Primary VLM evaluator with MMMU weight 53.1. Cited in Sections 3.3 and 5.

Available at:

—

[15] Üstün et al.\ — Aya Model (arXiv 2024)

Type: arXiv preprint \\ Link:

Summary: Presents the Aya instruction-fine-tuned multilingual model family from Cohere for AI. The Aya-Vision variant extends the model to multimodal inputs. With MMMU score of 39.9, Aya-Vision provides the lowest-weighted contribution to our ensemble but serves as a diverse third signal from a non-Meta/non-Alibaba architecture family, improving ensemble robustness.

Relevance: Third VLM evaluator with MMMU weight 39.9. Cited in Section 3.3.

Available at:

Benchmarks

[16] Yue et al.\ — MMMU Benchmark (arXiv 2024)

Type: arXiv preprint / NeurIPS 2024 \\ Link:

Summary: MMMU (Massive Multi-discipline Multimodal Understanding and Reasoning) is a benchmark of 11,500 college-level questions across 30 academic subjects requiring both visual and domain-knowledge reasoning. It is the most widely used calibration benchmark for VLM capability assessment. MMMU scores are reported as percentage accuracy: Qwen2.5-VL 53.1%, LLaMA-Vision 50.7%, Aya-Vision 39.9% at time of our paper.

Relevance: The basis for our VLM ensemble weights. Section 3.3 (Equation 1) uses MMMU scores as weights w_j in the composite scoring formula.

Leaderboard:

Prompting / Chain-of-Thought

[17] Wei et al.\ — Chain-of-Thought Prompting (arXiv 2022)

Type: arXiv preprint (NeurIPS 2022) \\ Link:

Summary: The seminal paper demonstrating that LLMs exhibit dramatically improved reasoning performance when prompted to "think step by step" before

producing an answer. CoT prompting is shown to be particularly effective for multi-step tasks where intermediate reasoning improves final output quality. The paper evaluates CoT on arithmetic, commonsense, and symbolic reasoning benchmarks using GPT-3 and PaLM.

Relevance: Cited in Section 3.2 as the basis for our Stage 2 CoT prompt design. The phrase "Think step by step through the component relationships before writing any PlantUML keyword" directly applies the CoT prompting strategy from this paper to UML code generation.

Statistics / Inter-Rater Reliability

[18] Fleiss — Measuring Nominal Scale Agreement Among Many Raters (1971)

Type: Journal article \\ Link:

Summary: Introduces Fleiss' Kappa ($ $), an extension of Cohen's Kappa to the case of multiple (>2) raters assigning categorical ratings. The formula accounts for chance agreement and produces a normalized score from -1 (complete disagreement) to $+1$ (perfect agreement). Fleiss' Kappa is the appropriate statistic when more than two evaluators independently rate the same items.

Relevance: Cited in Section 4.3 (Inter-Rater Reliability). Our 80-rater evaluation requires Fleiss' Kappa, not Cohen's Kappa. The reviewer comment about Fleiss vs.\ Cohen confusion is addressed by explicitly citing this paper.

—

[19] Cohen — A Coefficient of Agreement for Nominal Scales (1960)

Type: Journal article \\ Link:

Summary: Introduces Cohen's Kappa for measuring agreement between exactly two raters. This foundational paper establishes the concept of chance-corrected agreement statistics.

Relevance: Cited in Section 4.3 for completeness and to clarify that our study uses Fleiss' Kappa (for >2 raters) rather than Cohen's Kappa (for exactly 2 raters).

—

[20] Landis & Koch — The Measurement of Observer Agreement for Categorical Data (1977)

Type: Journal article \\ Link:

Summary: Proposes the widely used interpretation scale for Kappa values: <0: Less than chance; 0.01–0.20: Slight; 0.21–0.40: Fair; 0.41–0.60: Moderate; 0.61–0.80: Substantial; 0.81–1.00: Almost perfect. This scale is the standard reference for interpreting Fleiss' Kappa in software engineering and human-computer interaction research.

Relevance: Cited in Section 4.3 to interpret our $ =0.68$ as "substantial agreement."

Summary Table of All References

{p{0.4cm}p{3.5cm}p{2.0cm}p{2.0cm}p{5.5cm}}

\# & First Author (Year) & Venue & Access & URL \\

1 & Rumbaugh (2004) & Book & Purchase & \\ 2 & Fowler (2003) & Book & Purchase & \\ 3 & Arachchi (2021) & arXiv & Free & \\ 4 & Jahan (2024) & ICSE & Free & \\ 5 & Conrardy (2024) & arXiv & Free & \\ 6 & Gheorghita (2025) & ICSE & Free & \\ 7 & Bates (2025) & IEEE TSE & IEEE & \\ 8 & de Oliveira (2024) & CAiSE & Springer & \\ 9 & Chen (2023) & ACM CSUR & ACM DL & \\ 10 & Ambriola (2006) & ACM TOSEM & ACM DL & \\ 11 & Dubey (2024) & arXiv & Free & \\ 12 & Guo (2025) & arXiv & Free & \\ 13 & Gemma Team (2024) & arXiv & Free & \\ 14 & Wang (2024) & arXiv & Free & \\ 15 & Üstün (2024) & arXiv & Free & \\ 16 & Yue (2024) & arXiv/NeurIPS & Free & \\ 17 & Wei (2022) & arXiv/NeurIPS & Free & \\

18 & Fleiss (1971) & Psych.\ Bulletin & APA & \\ 19 & Cohen (1960) & Ed.\ Psych.\ Meas.

& SAGE & \\ 20 & Landis & Koch (1977) & Biometrics & JSTOR & \\

2.3 Gap Summary

Prior work often improves generation without scalable semantic validation, relies on a single judge model, focuses on one diagram family, or lacks a reproducible verified UML corpus. This thesis couples generation with MMMU-weighted multimodal ensemble verification and majority voting across four design-phase diagram types, and ships a public system that can be operated locally for demonstration and extension.

CHAPTER 3

METHODS AND SYSTEM DESIGN

Figure 1 (conceptual). Natural-language requirements enter Stage 1 (JSON specification). Stage 2 synthesizes PlantUML with reasoning. Stage 3 renders the diagram and applies VLM ensemble scoring with MMMU weights and majority vote. Failed renders receive S = 0 and do not enter VLM scoring.

Framework Overview

The framework operates as a sequential three-stage pipeline from raw natural-language requirements to a validated UML dataset. Figure illustrates the end-to-end flow.

The design principle is task decomposition: dividing the problem into smaller, analyzable stages reduces prompt complexity, simplifies debugging, and allows model-specific failures to be isolated at stage boundaries .

Stage 1: Technical Specification Generation

Input and output. The input to Stage 1 is a raw natural-language requirement string (e.g., a user story or a System Requirements Specification sentence). The output is a structured JSON object that enumerates candidate entities and their attributes, inter-entity relationships (association, aggregation, composition, dependency, realization), containment patterns, and diagram-type-specific constraints (e.g., which classes require interface stubs for a component diagram).

This JSON specification is the sole input passed to Stage 2, ensuring that the downstream model never sees raw, unprocessed requirement text.

Dataset source. The 8,000 technical specifications used in this study are generated entirely by the pipeline itself in an automated process with no manual authorship. To ensure domain diversity and reduce inter-specification repetition, the LLaMA

3.2-1B-Instruct model is prompted with a System Architect persona instantiated across four software application domains: e-commerce (online retail and inventory management), healthcare (patient record and appointment systems), IoT (smart-device telemetry and command processing), and banking (financial transaction and account management).

For each domain, the model generates 500 unique specifications per diagram type, yielding $4 500 4 = 8{,}000$ specifications in total. Each specification is uniquely identified by a domain-type-index triplet, and a domain-specific context sentence is injected at the start of each prompt to anchor the generated content to a realistic software context.

Model selection and justification. Stage 1 is handled by LLaMA 3.2-1B-Instruct . Because the primary goal of this stage is structured information extraction rather than complex multi-step reasoning, a lightweight instruction-following model is both sufficient and computationally efficient. LLaMA 3.2-1B-Instruct was selected for three reasons. First, its instruction-tuned variant reliably follows structured-output prompts — a property confirmed in recent LLM-for-SE studies that applied LLaMA-family models to requirements-analysis and specification-generation tasks .

Second, its compact parameter count (1B) keeps per-specification inference cost low, critical when processing thousands of requirements at dataset-construction scale. Third, it is fully open-licensed, ensuring complete pipeline reproducibility without API dependencies.

Alternative models considered. Two alternative open-source models were evaluated on 100 held-out requirement prompts (Table). Gemma 2-2B (Google) produced valid JSON in 82\,% of cases with an inference time of 3.2\,s per specification. DeepSeek-R1-Distill-Qwen-1.5B achieved 88\,% JSON validity at 2.1\,s. LLaMA 3.2-1B-Instruct outperformed both alternatives on JSON validity (94\,%)

while achieving the fastest throughput (1.8\,s), confirming it as the optimal specification generator for this pipeline.

Prompt decomposition rationale. Providing the downstream PlantUML generator with a pre-parsed JSON specification, rather than raw text, narrows the generation search space and improves output consistency. This prompt-decomposition strategy is grounded in Chain-of-Thought and multi-step prompting research, which shows that breaking complex tasks into sequential sub-tasks reduces error accumulation and improves reproducibility across model invocations .

Stage 2: PlantUML Synthesis

Input and output. The input to Stage 2 is the structured JSON specification produced by Stage 1. The output is a syntactically complete PlantUML script that encodes the target diagram type, all entities, relationships, containment structures, stereotypes, and annotations derived from the specification.

Why PlantUML. PlantUML is a text-based, scriptable, and version-friendly UML tool that integrates naturally with reproducible software engineering workflows. Its compiler provides a strict pass/fail render gate — any syntactically invalid script fails deterministically — which is exploited in Stage 3 to automatically exclude malformed outputs.

Why this task demands strong logical reasoning. Valid UML synthesis is substantially more demanding than structured extraction. The generator must correctly resolve multiple relationship types (association vs.\ aggregation vs.\ composition vs.\ dependency vs.\ realization), handle multi-level nesting for package and component diagrams, enforce correct PlantUML syntax for each relationship type, and maintain referential consistency across all named elements.

These requirements constitute a multi-step logical reasoning problem: the model must decompose the specification into an ordered sequence of entity declarations, infer

the correct connector type for each pair of entities, and ensure that no element is referenced before it is declared .

Model selection and justification. Stage 2 is handled by DeepSeek-R1-Distill-Qwen-32B . This model is specifically trained using reinforcement learning from verifiable feedback, a strategy that directly incentivizes multi-step logical inference — precisely the capability required for correct relational reasoning in UML synthesis. The RL-based reasoning alignment documented in the DeepSeek-R1 technical report yields strong performance on code generation and structured-output tasks even relative to models with substantially more parameters .

Its 32B parameter count provides the representational capacity needed for complex nesting and interface definitions, while the distilled variant from Qwen-72B keeps memory requirements compatible with consumer-grade GPUs (e.g., RTX 4090 with 8-bit quantization).

Chain-of-Thought prompting. Chain-of-Thought reasoning is enforced through <think> tags , instructing the model to decompose the input specification into named entities, select the appropriate UML connector for each entity pair, and validate the structural hierarchy before emitting any PlantUML keyword. This explicit step-by-step decomposition reduces syntactic errors by preventing premature code emission.

Diagram scope. The framework targets four design-phase diagram families:

• Class diagrams — classes, attributes, methods, and structural relationships (association, aggregation, composition, inheritance).

• Object diagrams — instance-level snapshots of the class structure at a specific runtime state, with concrete attribute values and associations.

• Component diagrams — modular system decomposition showing components, their provided and required interfaces, and inter-component dependencies.

• Package diagrams — containment, grouping, and inter-package dependency at the level of logical or architectural namespaces.

Stage 3: Rendering and Multimodal Verification

Render gate. PlantUML scripts are compiled using the official PlantUML engine . Any compilation failure immediately assigns a composite score of zero and excludes the sample from the VLM evaluation queue. This strict render gate ensures that only syntactically processable diagrams enter the semantic evaluation stage.

Four evaluation criteria. Passing diagrams are evaluated by three VLMs, each scoring the rendered diagram image against its source specification on a 0–6 integer scale across four criteria:

• Semantic correctness assesses whether the entities, relationships, and constraints described in the natural-language requirement are faithfully represented in the diagram. A diagram scores low if it introduces entities not mentioned in the requirement (hallucination) or omits entities that are explicitly required. Ambiguous or incorrect relationship types (e.g., modeling an aggregation as an association) are penalized here.

• Structural completeness assesses whether all major components mandated by the specification are present in the diagram. Unlike semantic correctness, which penalizes incorrect content, structural completeness penalizes omissions: a diagram that is correct where it does model something but leaves out half of the required elements receives a low completeness score.

• Syntactic accuracy assesses whether the diagram adheres to standard UML notation conventions for its specific diagram type. This includes correct use of arrow styles (open-diamond for aggregation, filled-diamond for composition, solid line for association, dashed line for dependency, hollow triangle for inheritance), correct multiplicity notation, proper stereotype usage ($ $interface$ $, $ $component$ $), and adherence to containment syntax for package and component diagrams.

• Overall coherence is a holistic quality criterion that assesses whether the diagram is logically organized, unambiguous, and usable as a communication artifact. A diagram may be semantically correct and structurally complete yet still score low on coherence if its layout is visually crowded, its

naming is inconsistent, or its relationships are difficult to

trace. This criterion captures practical communication value

beyond element-level correctness.

Vision-language model ensemble. Three open-source VLMs serve as independent evaluators: Qwen2.5-VL-3B-Instruct (MMMU score 53.1), LLaMA-3.2-11B-Vision-Instruct (MMMU score 50.7), and Aya-Vision-8B (MMMU score 39.9). These three models represent diverse trade-offs in parameter count, visual fluency, and reasoning depth, and their per-type performance varies in complementary ways, motivating an ensemble design over any single evaluator.

MMMU-weighted composite score. To reduce dependence on any single evaluator, individual scores are aggregated using weights derived from each model's MMMU benchmark performance . MMMU tests deep multimodal reasoning across 30 academic disciplines and serves as a principled proxy for diagram-interpretation ability. Let s_{ij} be the score assigned by VLM j to diagram i ($0 s_{ij} 6$), and let w_j be its MMMU accuracy weight ($w_1 = 53.1$, $w_2 = 50.7$, $w_3 = 39.9$).

An indicator function $ (s_i)$ excludes render-failed diagrams:

The final composite score is:

Worked example. Suppose VLMs assign scores $s_{i1}=5$, $s_{i2}=4$, $s_{i3}=3$ for a rendered class diagram ($ =1$): \[S_i = 1 {143.7} = {143.7} 4.09 \]

Majority-vote acceptance gate. In addition to the continuous composite score, a binary majority-vote gate provides an actionable pass/fail decision. Each VLM casts a vote based on acceptance threshold $ =4$ ("mostly correct with minor semantic gaps"):

A diagram is accepted if at least two of the three VLMs vote affirmatively:

This dual-signal design gives practitioners both a fine-grained quality indicator (the composite score) and an actionable accept/reject flag (the majority vote). A diagram enters the final dataset only if $A_i = 1$ and $S_i 3.0$.

Illustrative Example

Consider the requirement: "The system shall allow students to register for courses, instructors to manage course offerings, and administrators to monitor enrollment limits."

In Stage 1, LLaMA 3.2-1B-Instruct identifies key entities (Student, Course, Instructor, Administrator), operations (registration, offering management, enrollment monitoring), and constraints (enrollment capacity limit), encoding them as a JSON object. In Stage 2, DeepSeek-R1-32B uses Chain-of-Thought reasoning to map each entity to a UML class and each operation to an association or dependency, producing a PlantUML class diagram script.

After Stage 3 renders and verifies the diagram, the ensemble assigns scores (e.g., $s_1=5, s_2=5, s_3=4$), all three VLMs vote to accept, and the sample enters the validated dataset with composite score 4.70.

3.5 Scoring Equations (Paper-Faithful)

Let s_{ij} be the score of VLM j on diagram i on a 0–6 scale, and let w_j be the MMMU accuracy weights $w = (53.1, 50.7, 39.9)$ for Qwen2.5-VL-3B-Instruct, LLaMA-3.2-11B-Vision-Instruct, and Aya-Vision-8B respectively. Define $\delta_i = 1$ if diagram i passes the PlantUML render gate and $\delta_i = 0$ otherwise. The composite score is $S_i = \delta_i \cdot (\Sigma_j w_j s_{ij}) / (\Sigma_j w_j)$. When rendering fails, $S_i = 0$.

Each VLM casts vote $v_{ij} = 1$ if $s_{ij} \geq \tau$ with $\tau = 4$, else 0. The majority acceptance indicator is $A_i = 1$ if $\Sigma_j v_{ij} \geq 2$. A diagram enters the final dataset only if $A_i = 1$

and $S_i \geq 3.0$.

3.6 Worked Numerical Example

Suppose VLMs assign scores 5, 4, and 3 for a rendered class diagram ($\delta = 1$). Then $S = (53.1 \cdot 5 + 50.7 \cdot 4 + 39.9 \cdot 3) / 143.7 = 588.0 / 143.7 \approx 4.09$. Votes relative to $\tau = 4$ are yes, yes, and no, so majority accepts. Because $S \geq 3.0$, the sample is dataset-eligible. If PlantUML compilation fails, $\delta = 0$, $S = 0$, and the sample is excluded from semantic scoring and from the verified corpus.

Model	Params	Time (s)	JSON Valid (%)
LLaMA 3.2-1B-Instruct	1B	1.8	94.0
DeepSeek-R1-Qwen-1.5B	1.5B	2.1	88.5
Gemma 2-2B	2B	3.2	82.0

Table 1. Stage-1 model comparison on 100 requirement prompts (paper).

CHAPTER 4

IMPLEMENTATION AND SYSTEM ADVANCES

4.1 Software Architecture

The thesis deliverable includes a complete application stack. The backend is implemented with FastAPI and SQLModel for job and artifact persistence. The frontend is a Streamlit multipage UI covering single generation, batch orchestration, artifact review, human evaluation, analytics, and system design documentation. Versioned prompts live under prompts/. A provider factory selects among OpenAI-compatible APIs, Hugging Face routers, Ollama, mock providers for tests, and an MLX LoRA PlantUML adapter for local Stage-2 generation.

CLI scripts support corpus construction, evaluation, fine-tuning, and dual-Ollama setup. The public repository is https://github.com/dipak5501/uml-generation-pipeline.

4.2 Stage-1 JSON and Grounding

Stage-1 prompts request a JSON object with diagram_type, summary, entities, relationships, and optional packages, components, objects, and process_steps. The module app/services/spec_json.py parses and validates JSON, extracts named concepts from natural language, merges grounded entity names from requirement text or source-code analysis, and stores structured_json with validity metrics.

Prose derived from JSON is passed to Stage-2 prompts for readability while preserving paper-style structure. Invalid JSON triggers repair or fallback structuring rather than silent prose-only continuation when possible.

4.3 Stage-2 Generation, Validation, and Fidelity

PlantUML is generated with diagram-specific prompts and optional Chain-of-Thought. Outputs are sanitized, including repair of invalid worded arrows such as --inheritance--> into legal PlantUML connectors. Static validators reject empty diagram bodies and illegal package declarations. A fidelity gate compares PlantUML against Stage-1 entity and relationship coverage; weak or ModuleN-style outputs are replaced by a deterministic builder in plantuml_from_spec.py.

A repair loop attempts LLM fixes before assigning render failure. This design preserves the paper's render-gate philosophy while reducing professional-tester failures in which diagrams compile yet omit required domain entities.

4.4 Source-Code Mode and Script-Without-Types

In addition to natural-language requirements, the system accepts source code. Structural analysis recovers classes and relationships for Python, Java, and other languages when types are present. Critically, scripts that contain no class declarations (for example, a py2puml driver that only assigns variables such as source_folder and domain_module) must not invent classes from those variables.

entity_names() returns only real class declarations. When a class or object diagram is requested for such a script, the orchestrator redirects to a flowchart of the script process. This fix was validated after a professional fidelity review flagged false class diagrams for driver scripts.

4.5 Local Inference Stack

The paper Stage-2 model DeepSeek-R1-Distill-Qwen-32B is not run in-process on the student workstation. On Apple Silicon the stand-in is an MLX LoRA adapter on mlx-community/Qwen2.5-0.5B-Instruct-4bit. Production on the Math-department Mac Studio (Apple M1 Ultra, 128 GB unified memory) uses

FINETUNED_ADAPTER_PATH=models/uml-plantuml-lora-sourcecode-30k after 6,000 LoRA iterations on a Java/Python/C source-code corpus.

Dual Ollama daemons serve LLaMA-3.2-Vision-11B on Ollama 0.24 at :11434 and Qwen2.5-VL-3B on Ollama 0.32 at :11435; a single 0.32 process cannot load mllama. Aya-Vision-8B is loaded in-process with Transformers on MPS when unified memory is at least 64 GB (the 24 GB M2 hung on model.to(mps)). NVIDIA hosts retrain PEFT LoRA with scripts/finetune_plantuml_cuda.py and may serve Aya via vLLM on port 8001.

Environment variables ACCEPTANCE_TAU and MIN_COMPOSITE_FOR_DATASET encode the paper thresholds. Mock providers use the same fidelity builders so unit tests run without GPUs.

4.5.1 Production Mac Studio Deployment

The always-on server is a user-level LaunchAgent stack (no sudo): FastAPI on :8000, Streamlit on :8501, dual Ollama, caffeinate, and Cloudflare quick tunnels. A remote HTTP agent at /api/agent accepts allowlisted commands (health, server-status, generate, smoke-test, training-status, restart-api, restart-ui) with Bearer token auth. Public UI and API URLs are written to data/run/public_*.txt when tunnels restart.

Measured live generate on 2026-08-31 produced a class diagram with render success and composite S approximately 5.37 using the three-VLM ensemble. Keep the Mac user logged in (screen lock is permitted; Log Out stops LaunchAgents).

4.6 Paper–Code Alignment

Paper claim	Implementation
3-stage pipeline	app/services/orchestration.py
JSON Stage-1	spec_json.py + Stage-1 prompts

MMMU weights 53.1/50.7/39.9	settings + scoring
Render fail → S=0	verify_scores(render_ok=False)
Majority τ=4, S≥3.0	scoring defaults / .env
4 diagram types	class/object/component/package (flowchart training only)
Human 4 criteria	UI rubric (1–5) + paper 0–6 protocol
Mac Studio production	MLX 30k LoRA + dual Ollama + local Aya
NVIDIA path	PEFT CUDA LoRA + optional vLLM Aya

Table 4. Paper–implementation alignment summary.

4.7 Advances Beyond the Paper Baseline

• Source-code input mode with structural recovery (Python, Java, C).

• Flowchart/activity diagrams as an additional process view for scripts without types.

• MLX LoRA PlantUML on Apple Silicon (production adapter: source-code 30k).

• NVIDIA PEFT CUDA LoRA trainer and inference provider.

• Dual Ollama hosting for Qwen2.5-VL and LLaMA-Vision.

• Local Aya-Vision-8B on Mac Studio 128 GB (refuse in-process below 64 GB).

• Fidelity gate and deterministic PlantUML-from-spec builders.

• Script-without-types recovery (no fake classes from variables).

• Remote agent API and Cloudflare public tunnels for 24/7 demo.

• Package failure taxonomy, human evaluation UI, and analytics export.

CHAPTER 5

EXPERIMENTAL DESIGN AND RESULTS

5.1 Experimental Design

Research Questions

The empirical study is guided by three research questions:

• RQ1: How effectively can a decomposed LLM-based pipeline generate syntactically and semantically valid UML artifacts from natural-language requirements at scale?

• RQ2: To what degree does multimodal ensemble verification correlate with expert human evaluation of generated UML diagrams?

• RQ3: How does the accuracy of automated generation and validation vary across different UML diagram families, and what failure patterns explain the differences?

Dataset Scope

The design-phase dataset contains 8,000 UML samples, with exactly 2,000 samples per diagram type (class, object, component, package), generated across four software domains as described in Section III-B. For human evaluation, a stratified random sample of 40 diagrams was constructed by randomly selecting exactly 10 diagrams from each of the four diagram categories using a fixed random seed (seed $= 42$) to ensure reproducibility.

The choice of 40 samples is motivated by a formal statistical power analysis conducted using G*Power . Targeting a medium effect size ($f^{2}=0.15$) for correlation analysis at significance level $ =0.05$ and power $1{-} =0.80$ requires a minimum of 37 observations. The sample is rounded up to 40 for balanced stratification ($10 4$ types). A larger sample of 80 or 120 diagrams would increase statistical power marginally but would impose disproportionate annotation cost on 80 domain experts, each of whom must independently evaluate every sampled diagram.

Given that a sample of 40 already satisfies the power requirement for detecting medium-to-large effects, the 40-diagram threshold represents an effective trade-off between statistical adequacy and evaluator burden.

Human Evaluation Protocol

Human evaluation was conducted using a 0–6 integer scale identical to the VLM rubric, enabling direct score comparison. Expert raters assessed all four criteria (semantic correctness, structural completeness, syntactic accuracy, and overall coherence) for each sampled diagram. The evaluator pool consisted of 80 participants recruited across three groups: 23 university lecturers in information technology, 35 enterprise IT professionals with at least five years of software architecture experience, and 22 PhD students in software engineering.

All participants confirmed proficiency in UML modeling before being admitted to the study.

Inter-rater reliability was measured using Fleiss' Kappa ($ $) , the statistically appropriate measure for assessing agreement among three or more raters assigning categorical ratings simultaneously. Cohen's Kappa applies only to exactly two raters and was therefore not used . Kappa values are interpreted using the scale of Landis and Koch : $ [0.61, 0.80]$ indicates substantial agreement.

Correlation Analysis

To assess alignment between automated verification and human judgment, Pearson correlation coefficients (r) and Spearman rank correlations ($ $) between per-diagram VLM composite scores and average human scores are computed separately for each diagram type and in aggregate. p-values are reported to confirm statistical significance.

5.2 Paper-Scale Results (RQ1–RQ3)

RQ1: Effectiveness of Large-Scale UML Generation

How effectively can a decomposed LLM-based pipeline generate syntactically and semantically valid UML artifacts from natural-language requirements at scale?

Table reports render success rates, mean composite VLM scores, and standard deviations by diagram type.

Class diagrams achieve the highest render success rate (95.7\,%) and the highest mean composite score (4.31 out of 6.00, SD = 0.74). Object diagrams perform similarly well (success 94.4\,%, mean 4.09). Component diagrams show moderate difficulty (success 91.6\,%, mean 3.87, SD = 0.93). Package diagrams are the most problematic, with 379 render failures (18.9\,% failure rate) and the lowest mean score (3.12, SD = 1.04).

Analysis of failure logs reveals two primary failure modes for package diagrams: (1) incorrect PlantUML package nesting syntax (e.g., treating Pkg.Sub as two independent top-level packages rather than a nested structure), and (2) renderer memory overload on diagrams exceeding 50 nested elements. The majority-vote gate (A_i, Eq.) accepts 6,891 of 7,553 rendered diagrams (91.3\,%), providing a more conservative acceptance signal than the render gate alone.

Answer to RQ1: The decomposed pipeline achieves an overall render success rate of 94.4\,% and a majority-vote acceptance rate of 91.3\,% across 8,000 samples. Class, object, and component diagrams are generated reliably at scale; package diagrams require further work on hierarchical containment reasoning (81.1\,% success, mean score 3.12).

RQ2: Correlation Between Automated and Human Evaluation

To what degree does multimodal ensemble verification correlate with expert human evaluation of generated UML diagrams?

Table reports Pearson correlation coefficients and Fleiss' Kappa values by diagram type.

The overall Pearson correlation of $r = 0.71$ ($p < 0.001$) confirms strong positive alignment between VLM ensemble scores and human expert ratings. Class diagrams show the strongest correlation ($r = 0.82$, $ = 0.74$): both humans and VLMs consistently identify correct attributes, methods, and inheritance structures, and penalize the same syntactic errors. Object diagrams are close behind ($r = 0.76$, $ = 0.71$).

Component diagrams reach moderate correlation ($r = 0.68$, $ = 0.65$), with divergence arising primarily from dense visual layouts that impaired VLM visual parsing but were still interpretable by human experts. Package diagrams show the weakest correlation ($r = 0.55$, $p = 0.003$, $ = 0.58$), consistent with the ambiguity around containment-versus-dependency semantics in complex hierarchies.

The overall Fleiss' $ = 0.68$ falls in the substantial agreement range (0.61–0.80) , confirming that the four-criterion rubric supported consistent human judgment across lecturers, enterprise experts, and PhD students.

Answer to RQ2: Benchmark-weighted multimodal verification achieves $r = 0.71$ overall alignment with expert human evaluation (range: $r = 0.55$ for package to

$r = 0.82$ for class diagrams), validating it as a reliable and scalable proxy for manual quality control.

RQ3: Performance Variation Across Diagram Types

How does the accuracy of automated generation and validation vary across different UML diagram families?

Tables and jointly reveal a consistent performance gradient: class $>$ object $>$ component $>$ package across all metrics (render success, mean composite score, and human correlation). Three factors explain this ordering.

Training data availability. Class diagrams are by far the most common UML type in public code repositories and documentation, so both DeepSeek-R1-32B and the three VLMs have encountered more class diagram examples during pre-training, reducing generation and evaluation errors.

Syntactic rigidity. Class diagrams have the most constrained PlantUML grammar — a fixed set of keywords for classes, interfaces, attributes, relationships, and modifiers — reducing the surface area for generation errors. Package diagrams, by contrast, admit multiple competing syntactic constructs for the same semantic concept (containment vs.\ namespace reference), creating ambiguity for the generator.

Semantic ambiguity. Package diagrams conflate namespace, deployment, and architectural-grouping semantics in a single diagram type, creating confusion for both the generation model and the VLM evaluators. Component diagrams are intermediate: modular boundaries must be inferred rather than read directly from requirement text.

Answer to RQ3: Performance degrades systematically with diagram structural complexity. Class diagrams are the most tractable (95.7\,% render success, mean score 4.31, $r = 0.82$); package diagrams are the most difficult (81.1\,%, 3.12, $r = 0.55$), driven primarily by hierarchical containment complexity and PlantUML syntax sensitivity.

Comparison with Prior Approaches

Table positions this work against five prior LLM-based UML and architecture diagram generation approaches. The five approaches were selected as the most directly comparable from the literature search described in Section II.

Three advantages emerge from this comparison. First, the proposed pipeline achieves broader diagram coverage: four diagram types vs.\ at most two in any prior single work. Second, it provides stronger semantic verification: the benchmark-weighted ensemble ($r=0.71$) outperforms single-VLM approaches (e.g., de Oliveira et al., $r=0.61$) and eliminates the lack of automated verification seen in Jahan et al.

and Bates et al. Third, the scale of the validated dataset (8,000 samples) substantially exceeds all prior efforts (largest prior: 300 diagrams).

Diagram	Failures	Success %	Mean Score	SD
Class	87	95.7	4.31	0.74
Object	112	94.4	4.09	0.81
Component	169	91.6	3.87	0.93
Package	379	81.1	3.12	1.04
Overall	747	94.4	3.85	0.98

Table 2. Render success and mean composite score by diagram type (n = 2,000 per type).

Diagram	Pearson r	p-value	Fleiss κ
Class	0.82	<0.001	0.74
Object	0.76	<0.001	0.71
Component	0.68	<0.001	0.65
Package	0.55	=0.003	0.58
Overall	0.71	<0.001	0.68

Table 3. Automated vs human correlation and inter-rater reliability (40 diagrams, 80 evaluators).

5.3 Local Repository Validation

Using the public repository, scored Hugging Face subsets were analyzed locally. Package diagrams again show the lowest mean composite among available scored sets, consistent with RQ3. After fidelity hardening, a six-case grounded evaluation (NL library, hospital, checkout, banking, plus Python and Java source) achieved 6/6 MATCH with mean entity recall 1.0 and mean fidelity approximately 0.94.

Scripts without class declarations are redirected to process flowcharts rather than false class diagrams. These local checks do not replace the paper-scale 8,000-sample campaign, but they demonstrate that the released system behaves correctly under tester scrutiny on representative cases.

Case family	Result	Notes
NL class/object/component/package	MATCH	Entity names preserved
Python/Java with types	MATCH	Real classes only
Script without types	Flowchart	No fake classes

Table 5. Local fidelity check summary (post-hardening).

[Figure missing: vlm_scores_all.png]

[Figure missing: scores_package.png]

[Figure missing: scores_object.png]

[Figure missing: scores_component.png]

[Figure missing: scores_class.png]

5.4 Comparison with Prior Approaches

Relative to prior LLM-based UML or architecture diagram generators, the distinctive contribution is verification as a first-class component: MMMU-weighted ensemble

scoring, majority vote, and a hard render gate. Generation-only systems can report high syntactic success while still omitting required entities. Human-only evaluation does not scale to thousands of samples. This thesis therefore reports both render success and human-correlated multimodal scores, and ships tooling that rejects semantically weak but syntactically valid diagrams when fidelity coverage fails.

CHAPTER 6

DISCUSSION, LIMITATIONS, AND THREATS TO VALIDITY

6.1 Discussion

The dual-signal design (continuous S and binary majority acceptance) provides both ranking and inclusion decisions for dataset construction. Class diagrams are the most reliable family for both generation and evaluation; package diagrams remain the primary bottleneck due to nesting syntax and containment-versus-dependency ambiguity. Implementation advances such as the fidelity gate trade pure end-to-end LLM generation for professional correctness—especially important when models invent ModuleN placeholders or treat script variables as classes.

For CECS 698, the manuscript and the runnable system should be read together: the paper states the scientific protocol; the repository demonstrates operable engineering under resource constraints.

6.2 Limitations

- On-device Stage 2 uses Qwen2.5-0.5B LoRA rather than DeepSeek-R1-32B. Aya-Vision-8B runs locally on the 128 GB Mac Studio; 24 GB Macs must not load Aya on MPS.
- The VLM prompt currently requests a joint SCORE; the paper protocol discusses four 0–6 criteria, while the human UI uses 1–5 Likert items.
- Assembled local corpora are not identical to regenerating the paper's full domain × prompt matrix in one live run.
- Package diagrams remain the hardest family despite validators and builders.
- Behavioral UML (sequence/state) is out of scope for the verified 8k structural set.

6.3 Threats to Validity

Internal validity. The composite score weights are derived from MMMU benchmark accuracy, which measures general visual reasoning rather than UML-specific understanding. Future work should calibrate weights against a UML-specific benchmark.

External validity. The 8,000 specifications were synthetically generated and may not capture all ambiguities present in real-world industrial requirement documents. Benchmarking against Apache OSS project diagrams is an ongoing extension.

Construct validity. The 0–6 scoring rubric is operationalized consistently across VLMs and human raters, but the boundary between scores 3 and 4 was the most frequent source of human disagreement, particularly for component diagrams.

CHAPTER 7

CONCLUSION AND FUTURE WORK

This paper presented a three-stage automated pipeline for generating and verifying design-phase UML artifacts from natural-language requirements. By assigning LLaMA 3.2-1B-Instruct to structured specification generation, DeepSeek-R1-Distill-Qwen-32B to PlantUML synthesis, and an MMMU-weighted ensemble of three VLMs to multimodal verification, the pipeline produces 8,000 annotated UML samples with a 94.4\,% render success rate.

The dual-signal acceptance mechanism (weighted composite score $+$ majority-vote gate) yields both fine-grained quality indicators and actionable pass/fail decisions. Class diagrams achieve the strongest performance (95.7\,% success, mean score 4.31, $r = 0.82$ with human experts); package diagrams remain the most difficult (81.1\,%, 3.12, $r = 0.55$). Compared with five prior LLM-based approaches, the proposed pipeline achieves higher render success rates, broader diagram coverage, and stronger semantic alignment.

Future work will extend the pipeline to sequence and activity diagrams, integrate real OSS requirement corpora, and develop a UML-specific VLM calibration benchmark.

7.1 Future Work

- Behavioral UML (sequence and state) with the same multimodal verification stack.
- Stronger package repair with hierarchy-aware learning signals.
- Broader industrial case studies and human-in-the-loop rejection feedback.

• Full per-criterion VLM numeric heads (four scores) aligned to the human 0–6 protocol.

• Larger public release of verified triples on Hugging Face with documented licenses.

REFERENCES

[1] Ambler, S. W. (2004). The Object Primer (3rd ed.). Cambridge University Press.

[2] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.

[3] Brown, T., et al. (2020). Language models are few-shot learners. NeurIPS.

[4] DeepSeek-AI (2025). DeepSeek-R1 technical report.

[5] Faul, F., Erdfelder, E., Lang, A.-G., & Buchner, A. (2007). G*Power 3. Behavior Research Methods.

[6] Fowler, M. (2003). UML Distilled (3rd ed.). Addison-Wesley.

[7] McHugh, M. L. (2012). Interrater reliability: the kappa statistic. Biochemia Medica.

[8] PlantUML. https://plantuml.com/

[9] Qwen Team. Qwen2.5-VL technical report.

[10] Rumbaugh, J., Jacobson, I., & Booch, G. (2004). The Unified Modeling Language Reference Manual (2nd ed.).

[11] Vaswani, A., et al. (2017). Attention is all you need. NeurIPS.

[12] Viera, A. J., & Garrett, J. M. (2005). Understanding interobserver agreement: the kappa statistic. Family Medicine.

[13] Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. NeurIPS.

[14] Yue, X., et al. (2024). MMMU: A massive multi-discipline multimodal understanding benchmark. CVPR.

[15] Meta AI. Llama 3 model family documentation.

[16] Ustun, A., et al. (2024). Aya model family / Aya-Vision.

[17] Additional baselines and UML-generation papers are detailed in paper/literature_review.tex and paper/references.bib; expand this list into CSULB bibliography format before final submission.

APPENDIX A

PROMPT TEMPLATES AND CONFIGURATION

Versioned prompts live under prompts/: requirement_to_tech_spec.v1.txt, code_to_tech_spec.v1.txt, tech_spec_to_{class,object,component,package,flowchart}.v1.txt, vlm_scoring.v1.txt, repair_plantuml.v1.txt, and human_evaluation_rubric.v1.txt. Stage-1 requests JSON-only specifications. Stage-2 requires exact entity names and forbids ModuleN placeholders. VLM scoring lists the four paper criteria and requests SCORE/EXPLANATION format.

Environment templates are documented in .env.example, including ACCEPTANCE_TAU and MIN_COMPOSITE_FOR_DATASET.

APPENDIX B

REPOSITORY AND REPRODUCIBILITY

Clone https://github.com/dipak5501/uml-generation-pipeline. Create a virtual environment, copy .env.example to .env, then run make install and ./scripts/run_local.sh. UI: http://127.0.0.1:8501. API: http://127.0.0.1:8000. Health: /api/settings/health. Production Mac Studio uses MOCK_PROVIDERS=false, USE_OLLAMA=true, USE_FINETUNED_CODE=true, FINETUNED_ADAPTER_PATH=models/uml-plantuml-lora-sourcecode-30k, and VLM_AYA_BACKEND=local.

Remote operator: POST /api/agent/command with Bearer token. Unit tests: make test with MOCK_PROVIDERS=true. Dual Ollama: scripts/ensure_ollama_dual.sh. NVIDIA: make finetune-cuda (do not run on the Mac Studio). The gated class-scored Hugging Face dataset is optional and not required.

APPENDIX C

FIDELITY AND FAILURE ANALYSIS NOTES

Professional testing revealed that generation-only metrics (render success, high VLM scores) can mask semantic mismatches: missing required entities (e.g., Librarian), ModuleN placeholders, and treating py2puml script variables as classes. The fidelity gate and script-without-types recovery were added so UML developers and external testers see diagrams that match named entities in the input.

Package failures are further categorized (nesting, containment/dependency confusion, empty bodies) via package_failures analytics. Local report artifacts may be stored under data/eval/ (gitignored).

APPENDIX D

GLOSSARY

PlantUML: Textual language and compiler for UML diagrams.

MMMU: Massive Multi-discipline Multimodal Understanding benchmark used for VLM weights.

Composite score S: MMMU-weighted average of VLM scores; 0 if render fails.

τ (tau): Per-VLM acceptance threshold; paper default 4 on a 0–6 scale.

Majority vote A: Accept if at least two of three VLMs vote $s \geq \tau$.

Fidelity gate: Coverage check ensuring PlantUML reflects Stage-1 entities/relations.

LoRA: Low-Rank Adaptation; used for local PlantUML fine-tuned generation.

CECS 697 / 698: CSULB Independent Study sequence culminating in thesis.

End of draft. Before Thesis Office submission: paste into the official CSULB thesis template (Format Manual), expand bibliography from paper/references.bib, replace DRAFT markings, obtain committee signatures, and confirm length/structure with Dr. Yutong Zhao. Practical target discussed with the author: approximately 60–90 pages in official double-spaced format after expansion of figures and full reference list.